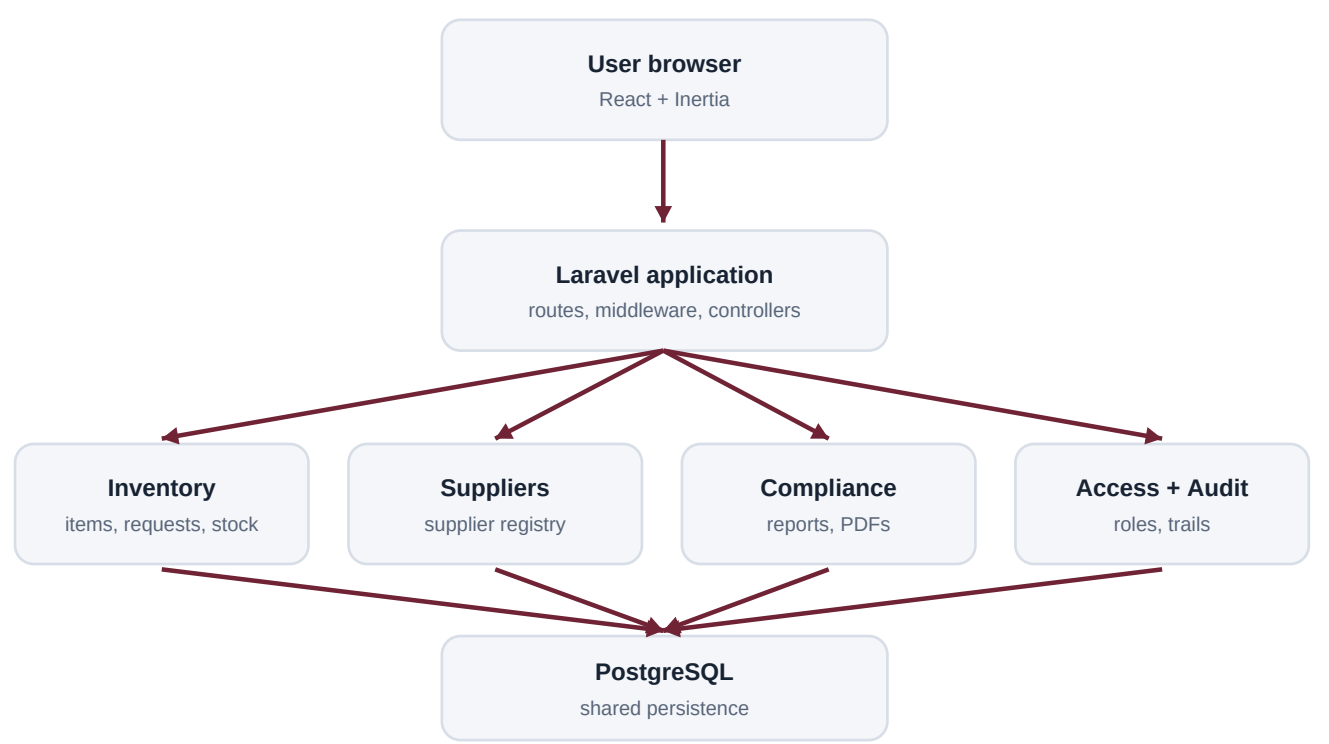


NEMIX Technical Manual

Architecture, installation, operations and support reference

Audience: maintainers, deployment operators and system administrators. This manual describes the repository state as of 03 Oct 2026. Validate environment specific values before production changes.

1. System architecture



NEMIX is a Laravel 12 modular monolith. Inertia connects server rendered route data to React/TypeScript pages. Modules include Inventory, Suppliers, AuditLogs and UserManagement. The shared database is PostgreSQL in the supplied Docker Compose configuration.

Layer	Implementation / location
Web routes and middleware	routes/web.php; module routes under Modules/*/routes/web.php
UI	resources/js/Pages and resources/js/utls/sidebarConfig.tsx
Business modules	Modules/Inventory, Modules/Suppliers, Modules/AuditLogs, Modules/UserManagement
Persistence	Laravel Eloquent models and database/migrations
Hardware	firmware/rfid-scanner and routes/api.php

2. Local installation

Setup sequence

- 1 **Copy .env.example to .env** configure local values
- 2 **Start app, queue and db** docker compose up -d
- 3 **Generate key and migrate** artisan commands
- 4 **Build assets and verify** npm run build / tests

1. Install Docker/Compose, then copy **.env.example** to **.env**. Set unique DB credentials, APP_URL and mail settings. Do not commit secrets.
2. Run **docker compose up -d --build**. The supplied Compose file binds the app to **127.0.0.1:8080**.
3. Run **docker compose exec app php artisan key:generate** and **docker compose exec app php artisan migrate**.
4. Build the front end with **npm run build**. Open the local app and verify sign in, dashboard and a permitted module.

The README and composer scripts contain older Sail service names and host settings. For this checkout, use docker-compose.yml as the local container reference (service **app**, DB host **db**).

3. Functional routes and ownership

Area	Primary route	Owner
Dashboard	/dashboard	core app
All Items	/inventories	Inventory
Receiving	/inventory/receiving	Inventory
Issuance	/inventory/issuance	Inventory
My Requests	/inventory/my-requests	Inventory
Suppliers	/suppliers	Suppliers
Compliance reports	/compliance/reports	core app
RFID workspace	/rfid-scanner	core app
Access Control	/access-control/*	core app
Audit Logs	/audit-logs/*	AuditLogs

4. Supply request transaction

State and stock effects

1	Pending	editable request; no reservation
2	Approved	reserves approved quantity
3	Issued	release creates issuance and deducts stock
4	Rejected / Cancelled	no release; reservation removed

1. SupplyRequestController receives submission from the coordinator. Store item IDs and requested quantities.
2. Approval records reviewer, time and RIS reference, and reserves the approved quantity. The physical item balance stays unchanged.
3. Release requires explicit signed RIS confirmation, rechecks stock inside a database transaction, creates a linked issuance and allocates FIFO batches.
4. A repeated release is rejected. Linked issuances cannot be independently edited or voided; a separate reversal workflow would be needed for post-release corrections.

5. RFID integration

Hardware request path

1	ESP32 + YRM100	read tag
2	HTTPS /api/hardware/rfid/*	configuration, scan, heartbeat
3	HMAC middleware	ID, timestamp, nonce, signature
4	Laravel controller and database	validate and record event

Provision the device in **System Settings > RFID**. Firmware setup uses a temporary access point and stores network, server URL, device ID and a one-time signing secret in ESP32 NVS. Hardware requests use X-Device-ID, X-Timestamp, X-Nonce and X-Signature headers. Review `firmware/rfid-scanner/README.md` before flashing or changing device configuration.

6. Security and access

1. Operational web pages use auth and verified middleware. Role and route permissions govern visible menus and controller actions.
2. Keep APP_DEBUG=false, secure session cookies, unique secrets and a working queue in production.
3. Keep PostgreSQL and pgAdmin off public interfaces. The supplied Compose mapping binds pgAdmin to loopback only when its admin profile is enabled.
4. Use an authorized backup process before migrations, and verify restore procedures separately from routine deployment.

7. Operations and verification

Task	Command or check
Container health	<code>docker compose ps</code>
Application logs	<code>docker compose logs -f app</code>
Queue logs	<code>docker compose logs -f queue</code>
Migration status	<code>docker compose exec app php artisan migrate:status</code>
PHP tests	<code>docker compose exec app php artisan test</code>
Front-end build	<code>npm run build</code>
Config refresh	<code>docker compose exec app php artisan optimize:clear</code>

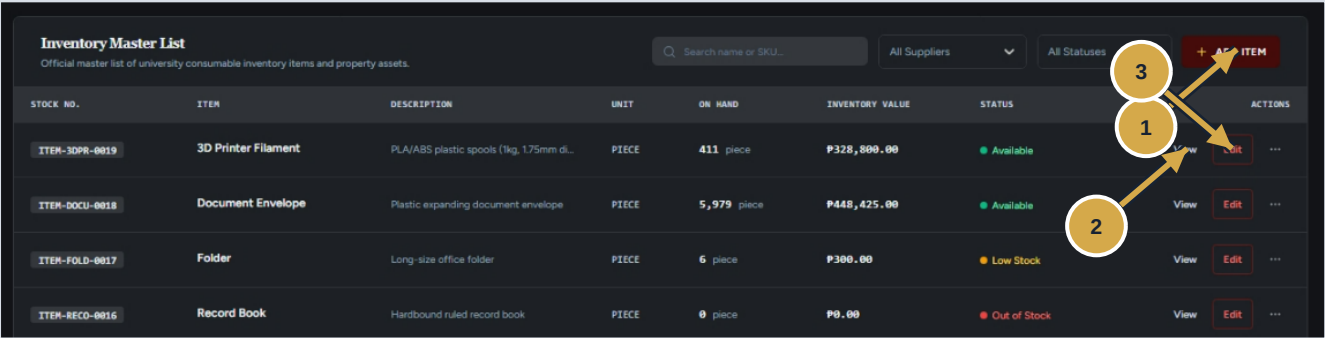
Deployment order: back up data; review migration impact; deploy code and built assets; run migrations; refresh caches; restart queue; check health and a representative receiving/request/report workflow. Use DEPLOYMENT_CHEATSHEET.md for the current host procedure, but verify every server specific value before execution.

8. Troubleshooting map

Failure	Inspect first
Page unavailable	Container status, routes, Apache log, APP_URL and proxy settings
Access denied	User role, permission mapping, auth and email verification
Stock mismatch	Receiving, issuance, reservation and linked request records
No email	Mail settings, queue worker and failed jobs
Scanner offline	Device Wi-Fi, heartbeat, time sync, HMAC identity and server TLS
Report discrepancy	Reporting period, source transactions and signatory settings

9. Screen-to-code trace

The following user-supplied screenshots provide a visual cross-check for maintainers. Numbered arrows point to UI functions and their implementation areas.



Marker / function	Route and implementation
1 Add Item	GET /inventories; resources/js/Pages/Inventory/AllItems; InventoryController
2 View	Inventory details modal and receiving batch data
3 Edit	Item master form; update action in InventoryController

10. Receiving implementation trace



Record Receiving

Register incoming inventory delivered to the university.

1

INVENTORY ITEM

Search and select an item...

2

SUPPLIER *

Deliveries can be received from any vendor

Select supplier...

3

SUPPLIER STOCK

AUTOMATIC

Generated automatically after selecting an item and supplier.

Automatically generated per receiving batch and retained for RPCI reporting.

QUANTITY RECEIVED	UNIT COST (P)	BATCH TOTAL (P)
e.g. 50	0.00	P0.00

4

DATE RECEIVED *

10/03/2026

Cancel

Save Receiving

Marker / function	Route and implementation
1 Inventory item	POST /inventory/receiving; InventoryController::storeReceiving
2 Supplier	Suppliers module relation on receiving
3 Quantity and cost	Receiving batch stock and valuation inputs
4 Save Receiving	Persists receipt and affects item stock

Source references

routes/web.php; routes/api.php; Modules/Inventory/routes/web.php; Modules/Suppliers/routes/web.php; Modules/AuditLogs/routes/web.php;
resources/js/utlis/sidebarConfig.tsx; docs/SUPPLY_REQUEST_WORKFLOW.md; firmware/rfid-scanner/README.md; docker-compose.yml;
DEPLOYMENT_CHEATSHEET.md.